MODULE 8 · PACKAGING & EXTENSIBILITY

Kustomize

Template-free customization: one base, many environments

WHY IT MATTERS

Dev and prod need different YAML — without forking it

Copy-pasting a manifest per environment works until someone edits the dev copy and forgets prod. Now the files drift and nobody trusts either one.

Kustomize keeps one shared **base** and layers small, declarative **overlays** on top — no templating language, no `{{ }}` placeholders. It ships built into `kubectl`.

BY THE END YOU CAN

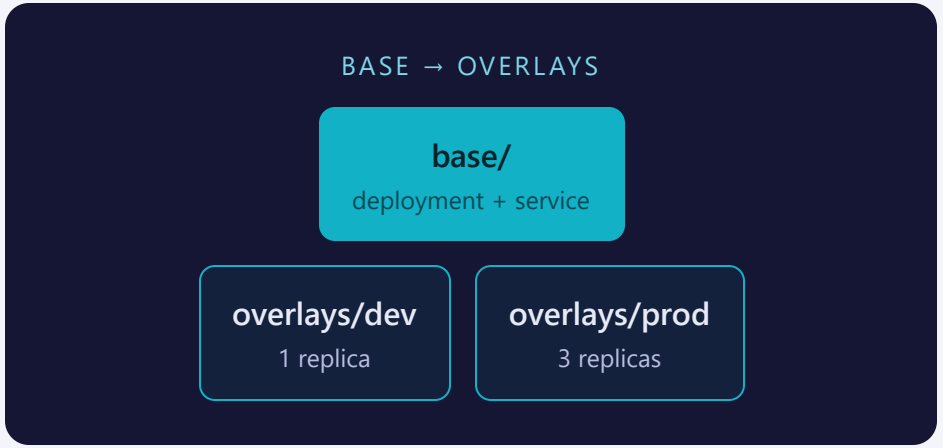
- Structure a base + overlay layout
- Patch replicas/images/labels per environment
- Preview with `kubectl kustomize`
- Apply with `kubectl apply -k`

CONCEPT

Template-free: patches, not placeholders

Helm generates YAML by substituting variables into templates. Kustomize does the opposite: it starts from **plain, valid YAML** and transforms it declaratively.

- **base/** — the shared bones: resources every environment needs
- **overlays/<env>/** — the deltas: namespace, labels, replicas, images
- Every directory Kustomize reads needs its own `kustomization.yaml`



BASE + OVERLAYS

The base is intentionally incomplete

BASE/	OVERLAYS/<ENV>/
Lists the real resources (`deployment.yaml` , `service.yaml`)	References the base: `resources: [../../base]`
Generic replica count, no namespace, no env label	Sets `namespace` , adds an `environment` label, patches replicas
Never applied directly to a real cluster	What you actually `apply -k`

RULE

Author base → overlay (bones first, deltas second). Apply overlay only — the base has no namespace and a placeholder replica count, so it represents no real environment.

MANIFEST ANATOMY

`kustomization.yaml` — the base

base/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml

labels:
  - pairs:
      managed-by: kustomize
    includeSelectors: true

configMapGenerator:
  - name: app-settings
    literals:
      - LOG_LEVEL=info
      - APP_GREETING=hello-from-kustomize
```

`includeSelectors: true` — `managed-by` is stable across every environment, so it belongs in the selector. `configMapGenerator` builds a `ConfigMap` named `app-settings-<hash>` and rewrites the Deployment's `envFrom` reference to that hashed name automatically.

MANIFEST ANATOMY

`kustomization.yaml` — the prod overlay

overlays/prod/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ../../base

namespace: training

labels:
  - pairs:
      environment: prod
    includeSelectors: false

patches:
  - path: replica-patch.yaml

images:
  - name: nginx
    newTag: "1.27"
```

The overlay's `environment: prod` label uses `includeSelectors: false`, adds `namespace`, and layers a `patches` file for `replicas`. The `images` transformer pins `nginx` to `1.27` for prod — no patch, no base edit; dev stays on the base tag.

DO IT — PREVIEW, THEN APPLY

Render first, apply the overlay only

PREVIEW — RENDER, DON'T TOUCH THE CLUSTER	APPLY — THE OVERLAY, NEVER THE BASE
`kubectl kustomize overlays/dev/`	`kubectl apply -k overlays/dev/`
Prints the fully merged YAML to stdout	Same merge, submitted to the API server

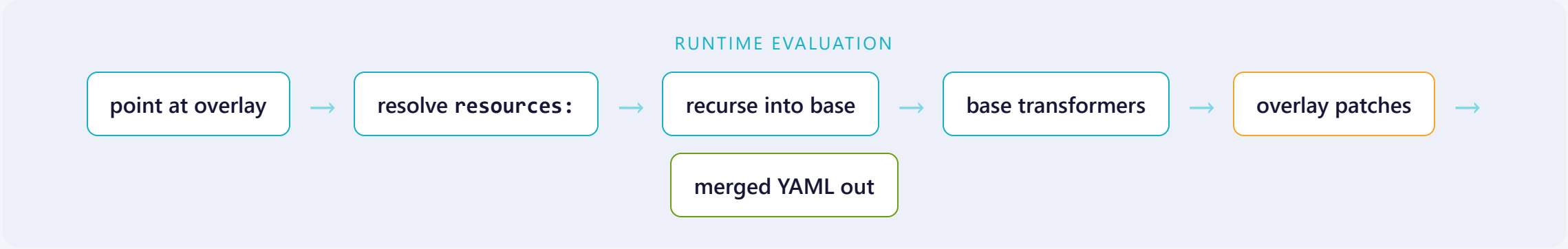
THE BRIDGE TRICK

Same imperative-vs-declarative idea as everywhere else in Kubernetes: inspect the rendered output before you commit to it.

```
$ kubectl kustomize overlays/prod/ | grep replicas # sanity check
$ kubectl diff -k overlays/dev/              # preview a re-apply
```

SEE IT

Resolution order: inside-out, then top-down



Apply dev (1 replica) then prod (3 replicas) against the same names — prod's `apply -k` simply overwrites dev's live objects.

PER-ENVIRONMENT KNOBS

Patches, images, replicas — pick the smallest tool

FIELD	WHAT IT CHANGES
`replicas`	Override a Deployment's replica count without a patch file
`images`	Override image name/tag/digest without editing the base container spec
`patches`	Strategic-merge or JSON 6902 edit to any field — the general-purpose tool
`labels` / `namespace`	Stamp metadata onto every resource the overlay pulls in
`configMapGenerator` / `secretGenerator`	Build a ConfigMap/Secret from literals or files — with a content-hash suffix

MENTAL MODEL

Reach for the dedicated field (`replicas`, `images`) before a raw `patches` entry — it's less to get wrong.

GENERATORS — THE SIGNATURE FEATURE

A content hash that rolls your pods

Why it matters. A hand-written ConfigMap keeps the same name, so editing it leaves running pods on stale config until you restart them by hand — the trap from Lab 3.1. A generated one gets a new hashed name on every change, so the Deployment's reference changes and pods roll on their own.

Housekeeping. Kustomize rewrites every reference to the hashed name and garbage-collects the previous ConfigMap on apply. `secretGenerator` works identically for Secrets.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Applying the base directly.** No namespace, no env label, placeholder replicas — it isn't a real environment.
- **Missing `kustomization.yaml`.** Every directory Kustomize walks needs one, or `apply -k` errors immediately.
- **Env label in the selector.** `.spec.selector` is immutable — putting `environment:` there blocks dev→prod forever.
- **Editing overlay output by hand.** Change the base or the patch, not the rendered YAML — it's regenerated every apply.

YOUR KUSTOMIZE TOOLKIT

Commands to keep at hand

```
$ kubectl kustomize DIR                 # render merged YAML, no apply
$ kubectl apply -k overlays/dev/         # build + apply an overlay
$ kubectl diff -k overlays/dev/          # preview a re-apply
$ kubectl get deploy -n training -o jsonpath='{.metadata.labels}'
$ kubectl delete -k overlays/prod/ --ignore-not-found # same names as dev
```

Tip: `kubectl kustomize` before `apply -k` — never guess what an overlay renders.

RECAP

Kustomize in one breath

- Base = shared bones; overlays = env-specific deltas, layered on
- Every directory needs its own `kustomization.yaml`
- Preview with `kubectl kustomize`, apply with `apply -k`
- Apply the overlay only — never the base

HANDS-ON

Now run `Lab 8.1`

`labs/8.1/`

Next: **8.2 — Helm**